01 · The system explained in plain language

Read this once before writing. It is accurate to the code and the running system as of 9 October 2026. Use it to write about CPUTrade confidently, in your own words.

1. What CPUTrade is

CPUTrade is a mobile-first online marketplace for the CPUT campus community. Students buy, sell and swap things (textbooks, electronics, clothes, services), approved outside vendors can sell too, and everyone can use a community bulletin board (events, study groups, Lost and Found). It installs on a phone like an app (a "progressive web app", PWA).

The problem it solves: students currently trade on WhatsApp and Facebook groups where nothing is verified, scams are easy, sales are not recorded, and nothing is built for campus life.

How it creates trust (this is the heart of the project):

1. Students must register with a CPUT email address.
2. Outside vendors cannot sell until an admin approves them.
3. A sale is recorded against a *named buyer*, and the **buyer must confirm they received the item** before they can rate the seller. A seller cannot rate their own sale, and nobody can rate twice. This blocks fake reviews.
4. Users can report and block others; admins can remove listings, suspend accounts and review appeals.

2. Who uses it (roles)

Role	How they join	What they can do
Student	Register with a CPUT email (`@mycput.ac.za` or `@cput.ac.za`)	Browse, list items, chat, buy, rate, wishlist, report, bulletin board
Vendor	Register with any email, **admin must approve**	Browse immediately; list and sell only after approval
Admin	Created by the team in the database; cannot self-register	Approve vendors, moderate listings, handle reports, suspend users, review appeals

3. Main features (what to describe)

- **Listings:** up to 6 photos (JPEG or PNG, 4 MB each), price, condition (New to Poor), category (Textbooks, Electronics, Clothing, Services, Other), campus (Bellville, District Six, Granger Bay, Mowbray, Wellington), quantity. Price R0 is shown as **Freecycle**. A listing can be marked **open to swap** with a note on what the seller wants.
- **Discovery:** search, category pills, campus filter, price range, "Open to Trade" filter, and a live "just sold" ticker. Item pages show views and how many people are watching.

- **Chat:** one private conversation per listing between buyer and seller, with read receipts, typing indicator, online/last-seen, edit and delete of your own messages, and safe meet-up location suggestions.
- **The sale flow:** seller picks the buyer from people who messaged them, then marks sold (for items with quantity above 1 it is "Sell One"); the buyer is notified and confirms receipt; then the buyer can rate 1 to 5 stars with a comment. A **Verified Seller badge** and average rating show on profiles.
- **Wishlist and saved searches:** heart an item to get alerts for a **price drop** or when it is **sold**; save a search to be told when matching items appear.
- **Notifications:** in-app notification centre plus **Web Push** to the phone. There are 8 notification types (vendor approved, listing removed, new message, price drop, wishlist item sold, saved-search match, confirm receipt requested, receipt confirmed).
- **Seller dashboard:** their listings, total views, items sold, average rating, unread chats.
- **Safety and moderation:** report a listing or user (scam, inappropriate, spam, harassment, other), block users, admin review queue, suspension (for a set time or indefinitely, effective on the very next request, lifted automatically when the time ends), and appeals.
- **Community bulletin board:** General, Lost and Found posts, filterable.
- **Platform:** installable PWA, offline cache of the feed, works on screens down to 320 px wide, share to WhatsApp.

4. How it works inside (simple three-layer picture)

```
Phone or laptop browser            Server                          Database
Vue 3 web app (PWA)   --HTTPS-->   Spring Boot REST API   --SQL-->  MySQL / MariaDB
screens, forms                     security, business rules         13 tables
(port 5173 in development)         (port 8080)                      (port 3306)
```

- **Front end (what users see):** Vue 3 with Vue Router (pages), Pinia (shared state), Axios (talks to the API), Vite (build tool), a service worker (offline and push). About 12 screens.
- **Back end (the brain):** Spring Boot 4 on Java 21. It receives requests, checks who you are and what you may do, applies business rules, and reads or writes the database. It exposes about **65 endpoints**.
- **Database:** MySQL/MariaDB. Hibernate creates and updates the tables from the Java classes.
- **Photos:** stored as files on the server, linked from the database.
- **Security chain on every request:** the login token (JWT) is checked; then the account is checked (suspended users and unapproved vendors are stopped); then the endpoint's role rule applies (admin pages are admin only).
- **Architecture style:** a **modular monolith**: one application divided into modules, not separate microservices.

Back-end modules

Module	Responsibility	Endpoints
user	Register, login	2
product	Listings, search, sold, photos, wishlist, saved searches	20
chat	Conversations and messages	10
trust	Reports, appeals, blocking	11
admin	Vendor approval, moderation, suspensions	8
bulletin	Community posts	4
notification	In-app notifications	4
push	Web Push subscriptions	3
profile	Public profiles, online status	2
rating	Ratings after a verified sale	1

5. The data (13 tables and how they connect)

`users` is the centre. Everything else hangs off users and products.

- `products` belong to a seller (user); a sold product can point to the buyer (`sold_to_user_id`); a product has many `product_images` .
- `conversations` link one product, one buyer and one seller; each has many `chat_messages` (with a sender).
- `ratings` link a product, the rater (buyer) and the ratee (seller); **one rating per product per rater** is enforced by the database.
- `wishlist_items` link a user and a product; `saved_searches` belong to a user.
- `notifications` have a recipient user; `push_subscriptions` belong to a user.
- `bulletin_posts` have an author; `reports` link a reporter to a reported user or product; `appeals` belong to a user; `user_blocks` link a blocker to a blocked user.

For an ER diagram, draw these 13 boxes and the lines above (the README has a diagram you can redraw).

6. Security and quality, in words you can use

- Passwords are stored **hashed with bcrypt**, never in plain text. Login tokens expire after 24 hours.
- **Rate limiting:** 5 wrong passwords for one username from one address locks that login for 15 minutes.
- **Roles and checks on every request**; admins cannot be created through registration.
- **Uploads:** only real JPEG/PNG files (checked by content, not file name), maximum 4 MB, stored under random names.
- **Secrets:** a production-mode startup check refuses to start with the development secrets.

- **Concurrency:** a database lock stops two simultaneous sales from corrupting stock (found and fixed during testing).
- **Tests:** 25 automated backend tests (all pass); a CI workflow written for GitHub; 15 manual scenarios.

7. How big is it?

About 5,800 lines of back-end Java, 280 lines of tests, 4,500 lines of Vue screens and components, 900 lines of JavaScript and 1,600 lines of CSS. Over 40 commits since 14 September 2026 (check with `git log --oneline`).

8. What is NOT in the system (and the honest reasons)

The brief asked for	Status	Reason you can give
Cart, checkout, payment gateway	Not built	Time and no merchant account; no payments means no card data is held, which removes the biggest breach risk; trade is in person with buyer confirmation
Escrow	Not built	No money is held; the buyer-confirmed receipt is a trust control instead
AI fraud detection, two-factor login	Not built	No transaction data to learn from; no high-value payments to protect; rule-based controls used instead
Real email verification	Not built	Needs an email provider; domain rule used instead
Microservices, Redis, cloud auto-scaling	Not built	Modular monolith is simpler and fits a three-person team and campus scale; stateless API can scale out later
Account deletion, privacy policy, penetration test	Not built	Out of scope for the time; listed as next steps

9. Tech and tools (for resource and quality documents)

Java 21, Spring Boot 4, Spring Security, JWT (jjwt), Hibernate/JPA, MySQL/MariaDB, Maven; Vue 3, Vite, Pinia, Vue Router, Axios, Workbox (PWA); JUnit 5 and Mockito for tests; Git and GitHub (including a CI workflow); VS Code and IntelliJ; Blackboard for submissions; free-licence stock photos (credits in the project).

10. Where to find proof

Evidence	Location
Screenshots of every main screen	`demo/screenshots/`
Run and demo instructions	`demo/PRESENTER-GUIDE.pdf`
Test code	`backend/src/test/java/` (run with `./mvnw test`)
Setup and deployment	`README.md`, `DEPLOYMENT.md`
Data and photo credits	`backend/db/`, `backend/uploads/products/CREDITS.txt`
History of work	`git log`